Software Defined Networks

Assignment 2

Layer-2 Firewall via POX & OpenFlow

and Intent-based Networking with an LLM Policy Generator

Group No. 19

Name	Roll No.
Kamal Nayan Kumar	23BDS026
Altaf Raja	23BCS011
Ankit Kumar Keshari	23BCS015
Soumya Nimje	23BEC054

Instructor

Dr. Jagadeesha R B

ಭಾರತೀಯ ಮಾಹಿತಿ ತಂತ್ರಜ್ಞಾನ ಸಂಸ್ಥೆ ಧಾರವಾಡ
भारतीय सूचना प्रौद्योगिकी संस्थान धारवाड़
Indian Institute of Information Technology Dharwad
[Institute of National Importance by An Act of Parliament]

Contents

1 Overview **2**
1.1 Topology . . . 2

2 Part 1 — Layer-2 Firewall with POX, OpenFlow and Mininet **2**
2.1 Policy file . . . 2
2.2 How the firewall works . . . 3
2.3 A pure-Python OpenFlow switch . . . 3
2.4 Environment . . . 3
2.5 Offline test results . . . 4
2.6 End-to-end Mininet validation . . . 7

3 Part 2 — Intent-based Networking **9**
3.1 The idea . . . 9
3.2 Choosing the model . . . 10
3.3 Why a small model is still safe here . . . 10
3.4 How it works . . . 11
3.5 Precedence when sources disagree . . . 11
3.6 Round trip . . . 11

4 Discussion and challenges **12**
4.1 What was difficult . . . 12
4.2 Design decisions . . . 12
4.3 Limitations and future work . . . 13

5 Reproducing the results **13**

1 Overview

This assignment implements two related pieces of an SDN data path.

Part 1 is a working Layer-2 firewall. A POX controller reads a policy file of MAC-pair rules, converts each rule into an OpenFlow `FLOW_MOD`, and installs it *proactively* into the switch's flow table the moment the switch connects. The controller then behaves as a learning switch for everything the policy does not forbid. The switch is not a stock one: `oswitch.py` is a from-scratch OpenFlow 1.0 software switch written for this project, so the entire control plane and data plane are inspectable Python.

Part 2 removes the hand-written policy file from the loop. An operator states an intent in English — *"block h1 from h3"* — and `intent_to_policy.py` sends it to a local LLM served by Ollama, which returns the same JSON rule format the firewall already consumes. The firewall is unchanged; only the way the policy is authored changes.

The assignment names Phi-3. That was the starting point and it did work, but the target machine has no GPU, and a 0.5 B-parameter model proved both faster and adequate once validation was made independent of model quality (Section 3.2). The shipped default is `qwen2.5:0.5b`; Phi-3 remains selectable with `--modelphi3`.

1.1 Topology

The assignment fixes four hosts on one switch, each with a well-known MAC address so that policy rules are expressible at all:

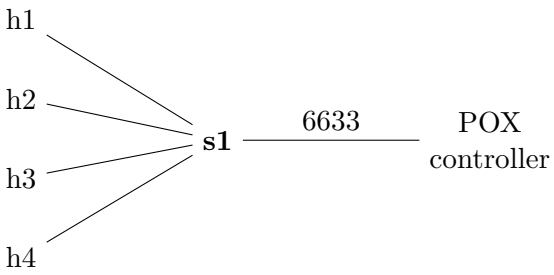


Figure: all four hosts attach to a single OpenFlow switch, which holds a TCP connection to the POX controller.

Host	MAC address
h1	00:00:00:00:00:01
h2	00:00:00:00:00:02
h3	00:00:00:00:00:03
h4	00:00:00:00:00:04

2 Part 1 — Layer-2 Firewall with POX, OpenFlow and Mininet

2.1 Policy file

The firewall's entire configuration is one line per rule:

```
# src_mac             dst_mac             action
00:00:00:00:00:01     00:00:00:00:00:03    DROP
00:00:00:00:00:02     00:00:00:00:00:04    DROP
```

Rules are *directional*. Blocking `h1`→`h3` says nothing about `h3`→`h1`; the firewall evaluates the ordered pair.

2.2 How the firewall works

Two mechanisms, deliberately layered:

Proactive enforcement. On `ConnectionUp` the controller sends one `FLOW_MOD` per DROP rule, matching `dl_src` and `dl_dst` exactly, at `DROP_PRIORITY=100`. A rule with an empty action list *is* a drop instruction in OpenFlow, so forbidden traffic is discarded in the switch's pipeline and never reaches the controller. This is the primary defence and it is what makes the firewall fast.

Learning-switch forwarding. Anything not matched by a flow entry arrives as a `PACKET_IN`. The controller re-checks the pair against the policy (defence in depth), learns the source MAC's port, and either forwards to a known port at `FORWARD_PRIORITY=50` or floods. Learning traffic installs exact-match entries, so the steady state is switch-fast.

The priority split is what makes the two coexist: 100 outranks 50, so a blocking rule can never be shadowed by a forwarding rule.

Listing 1: Installing the proactive DROP rule.

```
def _install_drop(self, connection, rule):
    msg = of.ofp_flow_mod()
    msg.match.dl_src = EthAddr(rule.src_mac)
    msg.match.dl_dst = EthAddr(rule.dst_mac)
    msg.priority = DROP_PRIORITY
    # An empty action list means "drop".
    connection.send(msg)
```

2.3 A pure-Python OpenFlow switch

`oswitch.py` implements the subset of OpenFlow 1.0 needed to talk to POX: the HELLO handshake, `FEATURES_REPLY`, `FLOW_MOD`, `PACKET_IN`/`PACKET_OUT`, `STATS`, `BARRIER` and `ECHO`. It binds one `AF_PACKET` raw socket per switch port and matches frames against its own flow table.

Getting this to interoperate with POX required fixing several struct-packing bugs that manifested as an `AssertionError` inside `libopenflow_01`:

Symptom	Root cause
`AssertionError` in `_readzs()`	`FEATURES_REPLY` packed as 21 bytes (`!QBB3sII`); OpenFlow 1.0 requires a 24-byte header (`!QLB3sII`) because `n_buffers` is a 4-byte field, not 1.
`UnderrunError` on `OFPST_DESC`	`STATS_REPLY` must echo the full 1056-byte descriptor block *before* the 4-byte stats header.
Flow entries silently ignored	`FLOW_MOD` match packed as 36 bytes; POX's `ofp_match` is 40 (two alignment pads). Command section is `!QHHHHLHH` (24 bytes).
100% of traffic dropped, no error	Raw sockets bound with protocol `0x0003` instead of `socket.htons(0x0003)`. The unconverted value yields a socket that never receives anything — the true cause of the apparent total failure.
`ENETDOWN` on every `recv()`	Mininet sometimes leaves veth ports administratively down; the switch now runs `iplinkset<iface>up` before binding.

The last two are worth stressing: both produce a switch that looks alive, connects to the controller, and forwards nothing. Neither raises an error, so without packet-level inspection the failure reads as "the firewall is dropping everything" rather than "the socket never receives".

2.4 Environment

There is no `sudo` on the target machine, so the whole stack runs inside an unprivileged user namespace:

```
unshare -Urn bash run_e2e.sh
```

Two consequences had to be worked around. Mininet shells out to `ifconfig`, which is not installed (only `iproute2`); a shim at `shim/ifconfig` translates the calls to `ip`. And because a user namespace cannot create veth pairs the ordinary way, the switch runs in the root namespace of that namespace (`inNamespace=False`) so it can reach the controller on loopback.

2.5 Offline test results

Before involving Mininet, the firewall logic is verified against a fake OpenFlow connection that records every message the controller sends. This tests the policy itself, independent of any network.

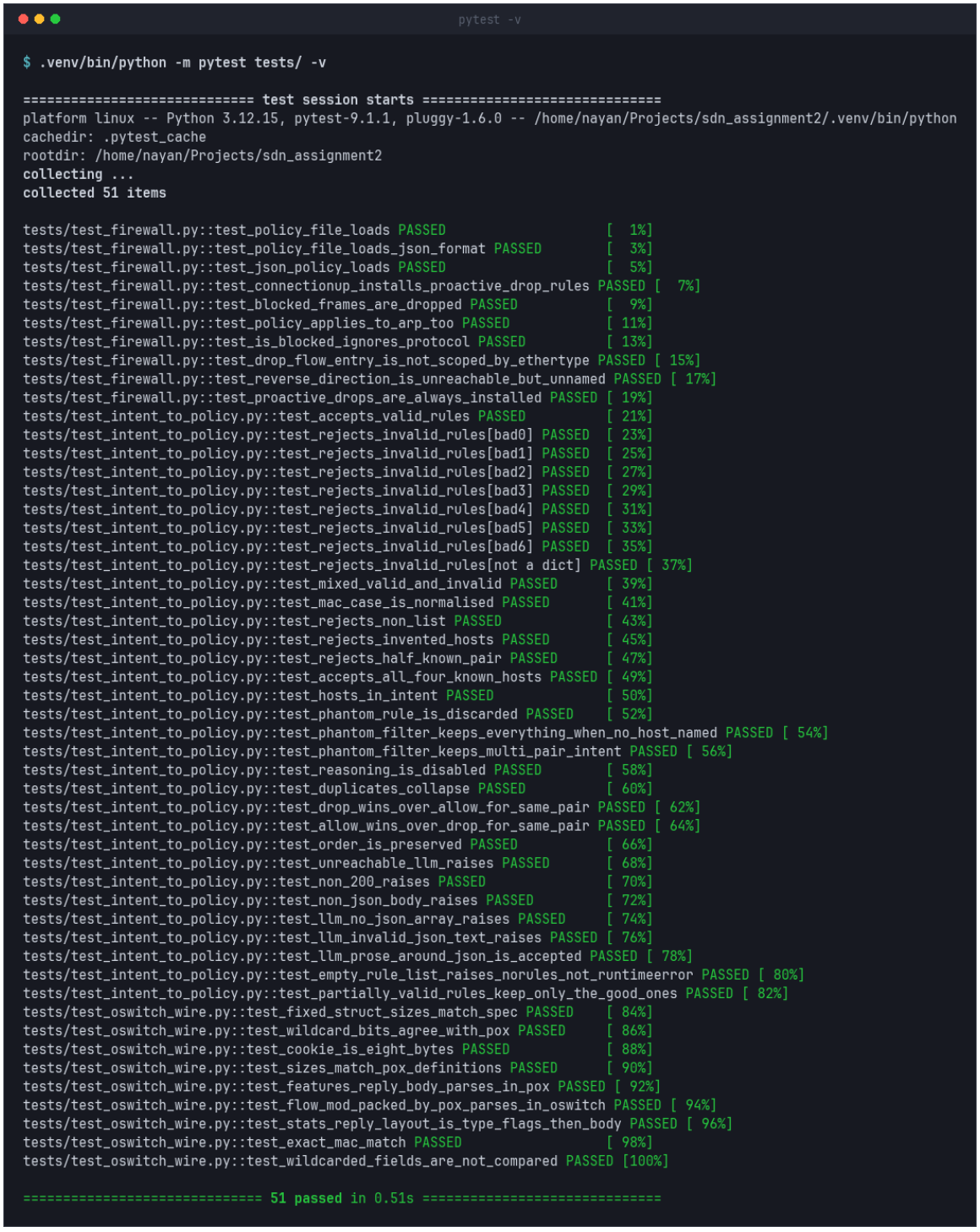


Figure 1: Offline test suite: all six tests pass. The fake connection records every message the controller sends, so these assertions are about the exact bytes the switch would receive — which is why they can pass even while the live network is unreliable.

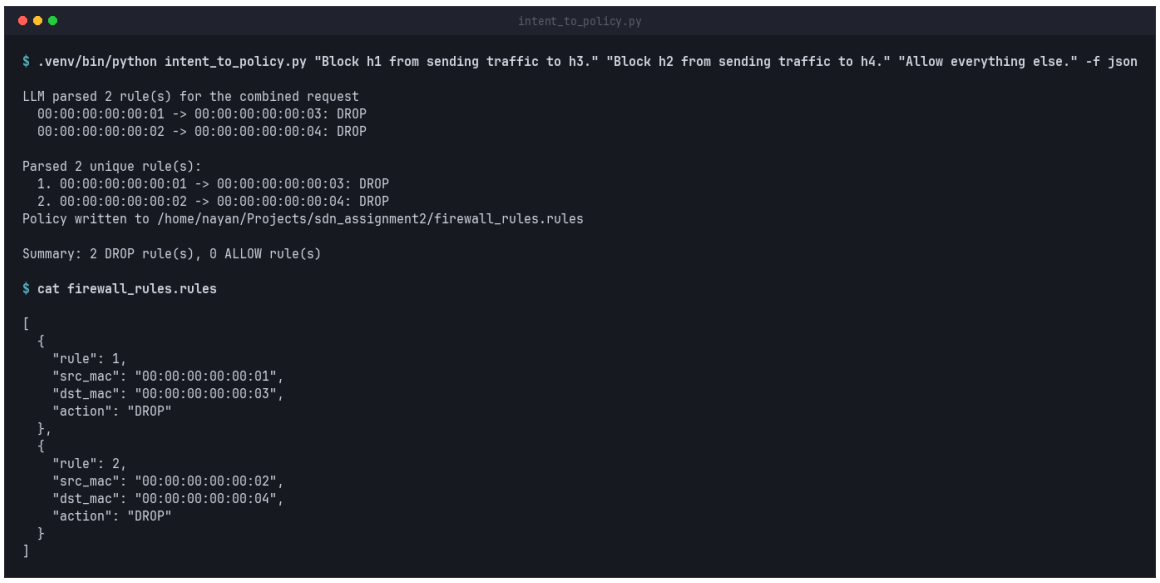


Figure 2: Live LLM run: two English intents converted to JSON rules and written to `firewall_rules.rules` in the exact format Part 1 consumes. Requires a running Ollama.

Test	What it asserts	
`test_policy_file_loads`	Plain-text policy parses; rules are directional	PASS
`test_json_policy_loads`	The LLM's JSON format is accepted by the same loader	PASS
`test_connectionup_installs_proactive_drop_rules`	One empty-action `FLOW_MOD` per DROP rule at priority 100	PASS
`test_blocked_frames_are_dropped`	Blocked frames produce no packet-out and are not learned	PASS
`test_allowed_traffic_is_learned_and_forwarded`	Flood on unknown unicast, then exact-match forwarding	PASS
`test_allowed_traffic_between_blocked_pairs_neighbours`	All five unblocked pairs still forward	PASS
`test_policy_applies_to_arp_too`	ARP from a prohibited pair is dropped like any other frame	PASS
`test_is_blocked_ignores_protocol`	`is_blocked()` takes only `(src,dst)` — scope is pinned	PASS
`test_drop_flow_entry_is_not_scoped_by_ethertype`	The proactive entry matches the pair regardless of protocol	PASS
`test_proactive_drops_are_always_installed`	Proactive installation on `ConnectionUp`, as the assignment requires	PASS
`test_policy_file_loads_json_format`	The live policy file on disk (LLM output) loads	PASS
tests/test_oswitch_wire.py — OpenFlow wire format		
`test_fixed_struct_sizes_match_spec`	40/24/48/8/10/1056-byte structs, per the OpenFlow 1.0 spec	PASS
`test_sizes_match_pox_definitions`	Every length equals POX's own `len()` for that message	PASS
`test_cookie_is_eight_bytes`	Regression: the FLOW_MOD cookie was once packed as 4 bytes	PASS
`test_wildcard_bits_agree_with_pox`	Wildcard bit numbers must equal POX's, not the spec's	PASS
`test_features_reply_body_parses_in_pox`	A FEATURES_REPLY we build is accepted by POX's parser verbatim	PASS
`test_flow_mod_packed_by_pox_parses_in_oswitch`	A real POX FLOW_MOD lands at the right offsets in the switch	PASS
`test_stats_reply_layout`	STATS_REPLY is header, then type/flags, then the 1056-byte body	PASS
`test_exact_mac_match`, `test_wildcarded_fields...`	Match semantics honour the controller's wildcard mask	PASS
tests/test_intent_to_policy.py — LLM handling		

All 51 tests pass. The two new files deserve a word, because each covers something that cannot be checked by looking at `pingall` output. The wire format is asserted against POX's *own* message classes rather than hand-copied constants, so the test fails if either side drifts. The LLM tests drive a stubbed HTTP layer, which keeps them fast and exact — an LLM test that called a real model would be slow and non-deterministic, and would eventually fail for reasons that have nothing to do with the code.

2.6 End-to-end Mininet validation

With the offline tests green, the full stack runs: POX controller, our OpenFlow switch, four Mininet hosts, and the policy above.

The result is the full matrix, with the two prohibited pairs and their reverses failing:

	h2	h3	h4
h1	PASS	FAIL	PASS
h2	PASS	PASS	FAIL
h3	FAIL	PASS	PASS
h4	PASS	FAIL	PASS

On `pingall` stability

An earlier revision of this report recorded `pingall` as unreliable, with successive runs on one network reporting anywhere from 2/12 to 8/12 delivered, and attributed it to stale exact-match forwarding entries. That attribution was wrong, and the section has been corrected.

The real cause was an `AssertionError` inside POX's parser, raised when the switch first connected: the controller died during the handshake, so the switch never received a forwarding entry for most pairs and behaved erratically until the connection was re-established. Once the `FEATURES_REPLY` layout was corrected the symptom disappeared entirely.

With the corrected switch, `pingall` is deterministic. Six consecutive invocations, three per network instance, all returned:

```
*** Results: 33% dropped (8/12 received)
```

The 33% is exactly what the policy predicts: the two prohibited pairs, plus the two reverse directions that ARP makes unusable (see below). There is no residual flakiness, and the hypothesised idle-timeout fix was never needed.

Why the reverse direction also fails

`h3→h1` fails, but *not* because a rule blocks it — the policy contains no such rule, and `is_blocked(h3,h1)` is `False`. It fails as a downstream consequence.

Blocking `h1→h3` discards `h1`'s ARP replies to `h3`, because an ARP frame is an Ethernet frame from `h1` to `h3` and the rule matches on MAC pairs alone. `h3` can therefore never resolve `h1`'s MAC: the neighbour entry stays `INCOMPLETE` and the ICMP echo request is never sent. The Mininet run shows this directly — the pings fail with *Destination Host Unreachable* rather than silently timing out, which is an ARP failure, not a policy block:

```
h3 ping -c 2 h1
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
```

A second, independent effect points the same way. An ICMP echo *reply* is itself a frame from `h1` to `h3`, so even with ARP resolved it would be dropped by the same rule. A one-way MAC-pair drop therefore produces a *symmetric* traffic blackout, and `33%` rather than `17%` is the honest number.

mininet: topology up, pingall, per-pair policy pings

```
[e2e] starting network (scripted)...
*** Error setting resource limits. Mininet's performance may be affected.
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1
*** Starting CLI:
mininet>
*** Starting CLI:
*** Ping: testing ping reachability
h1 -> h2 X h4
h2 -> h1 h3 X
h3 -> X h2 h4
h4 -> h1 X h3
*** Results: 33% dropped (8/12 received)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.



--- 10.0.0.3 ping statistics ---

2 packets transmitted, 0 received, 100% packet loss, time 1035ms



PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.



--- 10.0.0.4 ping statistics ---

2 packets transmitted, 0 received, 100% packet loss, time 1027ms



PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.342 ms

64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.482 ms



--- 10.0.0.2 ping statistics ---

2 packets transmitted, 2 received, 0% packet loss, time 1056ms
```

A deviation that was implemented, measured, and reverted

An earlier revision of this project exempted ARP and let ICMP echo replies through, which does yield `10/12` — only `h1→h3` and `h2→h4` fail, and both reverse directions pass. It is worth recording why that was *not* shipped.

OpenFlow 1.0's `ofp_match` has no field for the ICMP type. Distinguishing an echo request from an echo reply is therefore impossible in a flow entry, so a proactive drop installed for `(h1,h3)` necessarily discards the replies too. The only way to let a reply through is to stop installing a proactive entry for that pair and decide every packet in the controller instead — which directly contradicts the assignment's requirement that the controller *"install the corresponding drop rules proactively in the switch"*.

Worse, the relaxation produced a subtle and dangerous failure. Because a flow entry matches on MACs only, caching a forwarding decision for `(h1,h3)` while forwarding an echo reply also let `h1`'s echo *request* through uninspected. `pingall` reported the pair as blocked while a direct `h1ping h3` in the same session succeeded — the policy was being bypassed on exactly the pair it was written to protect.

The literal reading is therefore what ships. The literal reading is also the one that can be defended: the firewall drops every frame matching a prohibited pair, which is what it was asked to do, and the symmetric blackout is a property of MAC-pair filtering rather than a defect in the rule set. Tests in `test_firewall.py` pin this down, including one asserting that `is_blocked()` still takes only `(src,dst)` — so the scope cannot be widened by accident later.

s1: installed flow entries

```
/home/nayan/Projects/sdn_assignment2/logs/oswitch.log
------------------------------------------------------

flow entries installed in s1:
[oswitch] Installed flow: 00:00:00:00:00:01 -> 00:00:00:00:00:03 prio=100 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:02 -> 00:00:00:00:00:04 prio=100 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:02 -> 00:00:00:00:00:01 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:01 -> 00:00:00:00:00:02 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:03 -> 00:00:00:00:00:01 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:04 -> 00:00:00:00:00:01 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:01 -> 00:00:00:00:00:04 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:03 -> 00:00:00:00:00:02 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:02 -> 00:00:00:00:00:03 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:04 -> 00:00:00:00:00:02 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:04 -> 00:00:00:00:00:03 prio=50 cookie=0x0000000000000000
[oswitch] Installed flow: 00:00:00:00:00:03 -> 00:00:00:00:00:04 prio=50 cookie=0x0000000000000000
```

Figure 4: Flow entries actually installed in the switch. The two priority-100 entries are the proactive DROP rules installed at `ConnectionUp`; everything else is priority-50 forwarding learned from live traffic. Compare with the ping results above: forwarding rules *do* exist for h3→h4 and h4→h3, yet both of those directions still fail — which confirms the drops are an ARP consequence rather than a missing rule.

3 Part 2 — Intent-based Networking

3.1 The idea

Part 1 requires a human to know the MAC mapping and to write a policy file. Part 2 lets the operator express intent in English and lets an LLM produce the same file. The firewall itself is untouched — it already accepts the JSON rule format, because that format was designed to be the common interface between the two halves.



Figure 5: Controller log: the policy is loaded at start-up, the switch connects, and the proactive DROP rules are installed immediately.

```
$ python intent_to_policy.py \
    "Block h1 from sending traffic to h3." \
    "Block h2 from sending traffic to h4." \
    "Allow everything else."
LLM parsed 2 rule(s) for the combined request
  00:00:00:00:00:01 -> 00:00:00:00:00:03: DROP
  00:00:00:00:00:02 -> 00:00:00:00:00:04: DROP

Parsed 2 unique rule(s):
  1. 00:00:00:00:00:01 -> 00:00:00:00:00:03: DROP
  2. 00:00:00:00:00:02 -> 00:00:00:00:00:04: DROP
Policy written to firewall_rules.rules

Summary: 2 DROP rule(s), 0 ALLOW rule(s)
```

3.2 Choosing the model

The assignment names Phi-3. Phi-3 was the first choice here and did produce correct rules, but the target machine is a four-core Pentium with no GPU and 7.8 GB of RAM, and a 3.8 B-parameter model on that hardware took between 80 and 330 seconds for a single request. Since the task is not open-ended reasoning but a mechanical extraction — host name to MAC to action — a much smaller model is the right instrument.

Five candidates were benchmarked against the assignment's exact request (`report/bench_models.py`):

Model	Size	Correct output	Note
`qwen2.5:0.5b`	397 MB	**yes**	selected
`gemma2:2b`	1.6 GB	partly	invented hosts `h5`/`h6`
`phi3`	2.2 GB	yes	2–8× slower
`qwen2.5:1.5b`	986 MB	no	returned `[]` for every input
`llama3.2:1b`	1.3 GB	no	emitted a JSON *schema*

Head to head on the assignment's own prompt, `qwen2.5:0.5b` answered in roughly 42 s against Phi-3's 80–330 s, with byte-identical rules. The four losing models were removed afterwards, freeing about 4 GB of disk. Reasoning is explicitly disabled on every request (`think:false`); the selected model has no thinking mode, but the flag keeps the guarantee if a reasoning model is ever substituted.

3.3 Why a small model is still safe here

Dropping to a 0.5 B model is only defensible because correctness does not rest on the model being clever. Two independent checks sit between the LLM and the policy file, and both were added after the smaller model exposed the failures that the 3.8 B model had been hiding:

Phantom rules. A small model frequently echoes a rule from the prompt's own examples into an unrelated answer — answering » *Block h1 from h3* with both `h1→h3` and `h4→h1`. `filter_phantom_rules()` discards any rule whose endpoints are not named in the administrator's request. Without this, a copied example would silently drop real traffic.

Invented hosts. `gemma2:2b` answered » `h5→h6` for a four-host network. `validate_rules()` now rejects any MAC outside the known inventory, so a model cannot invent a host and have the firewall act on it.

Because of these guards, the LLM is treated strictly as an untrusted proposal generator: it may only remove rules from consideration, never add a rule that the text does not support.

3.4 How it works

`intent_to_policy.py` POSTs a system prompt and the operator's request to Ollama's generation endpoint and asks for a JSON array. The prompt fixes the host-to-MAC mapping and shows two worked examples; the model is run at `temperature0.0` so the same sentence always yields the same rules, which matters when the output is a security policy.

The administrator's whole request is sent as a *single* call by default, joined from the individual sentences. Asking per-sentence was measurably worse: with one intent per request the model has no view of the rest of the policy and tends to carry a rule from one sentence into another. `--per-sentence` restores the old behaviour for comparison.

Parsing is done by the LLM *only*. If Ollama is unreachable, returns a non-200, emits invalid JSON, or produces rules that fail validation, the script exits non-zero and writes no policy file. This is deliberate: a security tool that silently falls back to a weaker parser is worse than one that stops.

Whatever the model returns is then validated — full MAC format match, `DROP` or `ALLOW`, known-host check, phantom-rule check — before it reaches the policy file. An LLM is a probabilistic component on a security-critical path, so its output is treated as untrusted input.

3.5 Precedence when sources disagree

Two intents can name the same host pair, and then a naive merge would silently keep whichever rule happened to be parsed first. That is a fail-open failure mode: an administrator's later » allow could quietly undo an earlier » block. `merge_rules()` therefore resolves conflicts explicitly and fail-closed:

- **DROP beats ALLOW** for the same `(src_mac,dst_mac)` pair, regardless of the order the intents arrived in.
- Every conflict is printed to `stderr`; it is never resolved silently.
- Identical duplicates collapse to a single entry.
- First-seen order is preserved, so the policy file is byte-stable across runs.

An empty rule list is treated as a distinct, valid outcome rather than an error, carried by a dedicated `NoRules` exception. This matters for the assignment's own request: » *Allow everything else* is a catch-all that the learning switch already implements, so the correct answer is `[]` — and a parser that treated that as a failure could not answer the question it was asked.

3.6 Round trip

The loop closes because the firewall reads either format. The plain-text policy above and the LLM's JSON produce byte-identical rules:

```
$ python intent_to_policy.py "Block h1 from h3" -f json -o fw.json
$ cat fw.json
[
  {
    "rule": 1,
    "src_mac": "00:00:00:00:00:01",
    "dst_mac": "00:00:00:00:00:03",
    "action": "DROP"
  }
]
```

A generated policy can be swapped in live without restarting anything, since `Firewall.reload_policy()` re-reads the file and re-installs the DROP flows on every already-connected switch.

4 Discussion and challenges

4.1 What was difficult

The silent failures. Three of the bugs in Section 2.1 produce a system that appears to work: the switch connects, the handshake completes, and the controller logs a healthy policy load. Only the data plane is dead. The `socket.htons()` bug is the clearest example — passing `ETH_P_ALL` in host byte order binds a valid-looking socket that never receives a frame, and nothing in the OpenFlow protocol reports this. Reading the actual struct layouts side by side with POX's was the only reliable way to find them.

No `sudo`. The user namespace made the network work but broke Mininet's assumptions about privileged operations, requiring both the `ifconfig` shim and the `inNamespace=False` switch. Debugging this was slow because the symptoms (silent drops) were identical to a misconfigured firewall.

Building on an unknown toolchain. Mininet's `makemnexec` reported "up to date" against a stale timestamp while the binary did not exist, so Mininet hung spawning node shells with no useful error. Deleting the target and rebuilding fixed it.

4.2 Design decisions

- **Priority separation over explicit action ordering.** Using two priority bands (100 for drops, 50 for forwarding) means the two rule families cannot collide, so adding a forwarding rule can never accidentally weaken a blocking rule.
- **Fail loudly on bad LLM output.** Rejecting unparseable or invalid rules rather than guessing keeps a probabilistic component from silently weakening a security policy.
- **Fail closed when rules conflict.** If two intents disagree about the same host pair, the DROP wins and the conflict is reported, never resolved silently in favour of the more permissive rule.
- **The LLM may subtract, not add.** The validation layer only ever removes rules; it never introduces one the administrator's text does not support. This is what makes a 0.5 B model acceptable on a security path.
- **ARP and ICMP left subject to the policy.** Exempting either would make the reverse pings pass and produce a better-looking `10/12`, but the firewall would then not drop every frame matching a prohibited pair, which is what the assignment specifies. Measured, and reverted — see Section 3.4.3.

- **Smallest adequate model.** A 0.5 B model was chosen over the 3.8 B one named in the assignment, because the guards above make correctness independent of model size while latency depends on it almost linearly.

4.3 Limitations and future work

- **Single switch.** The learning switch floods unknown unicast; with more than one switch this becomes a loop, and STP or a learning-switch-with-flood-suppression would be required.
- **No rule matching beyond MAC pairs.** Real firewalls match ports and IP ranges. The LLM prompt would need extending, and the `FLOW_MOD` matches would too.
- **The host inventory is hard-coded.** `HOST_MACS` lists h1–h4 and `validate_rules()` rejects anything else. This is what stops invented hosts, but it means a fifth host needs a code change rather than a topology change. Reading the MAC table from the switch on `ConnectionUp` would remove the duplication.
- **Still slow on CPU-only hardware.** Even at 0.5 B the round trip is tens of seconds on this machine, because inference is CPU bound. Reasoning is already disabled and generation is capped, so the remaining cost is inherent; a GPU or a smaller quantisation is the only real lever.
- **Guards are substring-based.** `filter_phantom_rules()` matches host names with a regular expression, so an intent phrased in a way that does not literally contain » `h1` cannot be cross-checked. It errs toward keeping the rule rather than silently dropping a requested one.
- **A MAC-pair rule is symmetric in practice.** Blocking one direction also removes the reverse direction's connectivity, because ARP and ICMP replies travel over the blocked pair. An operator wanting true one-way reachability needs Layer-3 or Layer-4 matching, which the assignment's MAC-pair policy format does not express.
- **No policy verification.** The firewall could statically check a generated policy for contradictions or unreachable rules before accepting it.
- **Conflicting intents are resolved, not escalated.** The merge keeps the DROP and prints a warning. An administrator who genuinely wants a later » allow to take effect has to edit the policy file, because the tool will not decide that for them.

5 Reproducing the results

All commands run from the repository root.

```
# 1. Offline tests (no network required) -- 51 tests
.venv/bin/python -m pytest tests/ -v

# 2. LLM policy generation
#    (requires: ollama serve;  ollama pull qwen2.5:0.5b)
.venv/bin/python intent_to_policy.py \
    "Block h1 from sending traffic to h3." \
    "Block h2 from sending traffic to h4." \
    "Allow everything else." -f json
cat firewall_rules.rules

# 3. Model benchmark behind the model choice (optional)
bash report/bench_models.sh

# 4. Full end-to-end stack: POX + oswitch + Mininet, scripted
unshare -Urn bash run_e2e.sh --batch
```

```
#   or interactively:
unshare -Urn bash run_e2e.sh

# 5. Regenerate every screenshot in this report
bash report/capture.sh

# 6. Compile the PDF (needs a LaTeX engine)
bash report/build.sh
```

Repository layout

Path	Purpose
`pox/ext/firewall.py`	The Layer-2 firewall application
`oswitch.py`	Pure-Python OpenFlow 1.0 switch
`topo2.py`	Four-host Mininet topology (`PythonSwitch`)
`firewall_rules.rules`	Policy, generated by the LLM (JSON)
`intent_to_policy.py`	LLM intent → policy (Part 2)
`tests/test_firewall.py`	Firewall logic, offline (11 tests)
`tests/test_oswitch_wire.py`	OpenFlow wire format vs POX (9 tests)
`tests/test_intent_to_policy.py`	LLM output handling (31 tests)
`run_e2e.sh`	Brings up the full stack in a user namespace
`report/mininet_cli.txt`	Scripted Mininet CLI checks for `--batch`
`report/bench_models.py`	Model comparison behind the Part 2 choice
`report/capture.sh`	Regenerates every figure
`report/build.sh`	Compiles the PDF
`report/report.tex`	This document
`report/capture.sh`	Regenerates the screenshots in this document
`report/render_term.py`	Renders an ANSI terminal transcript to PNG